

Spaces - Technical Documentation

Employee community events platform for Manulife Philippines. Prepared for pitch review.

1. What the system does

Spaces lets Manulife Philippines employees discover, join and run internal community events - volunteer drives, wellness sessions, sports and networking meetups. Events are either **on-site** (mapped with a real coordinate) or **virtual** (a meeting link). Attendance earns rewards, which roll up into a leaderboard.

Three roles, one codebase:

- **user** (employee) - browses approved events, registers, cancels, views own rewards.
- **organizer** - everything a user can do, plus creates and edits own events and views their attendee lists. Employees apply for this role; an admin approves.
- **admin** - approves or rejects submitted events, manages users and roles, views analytics.

Two governance gates define the product: every event needs admin approval before it is visible or joinable, and every organizer needs admin approval before they can publish.

2. Tech stack

Layer	Choice	Why
Frontend framework	React 19	Component model; current stable major
Build tool	Vite 8	Fast dev server + HMR; ES module native
Routing	React Router 7	Nested routes, layout routes, route guards
Styling	Tailwind CSS 4 + daisyUI 5	CSS-first config via @theme tokens; Material 3 design tokens defined once in index.css
Maps	Leaflet + react-leaflet	Open source, no API key, no per-load billing
HTTP client	axios	Request interceptor attaches the JWT in one place
Linting	oxlint	Rust-based, near-instant on this codebase size
Backend runtime	Node.js (ESM)	Same language as frontend; one mental model
Backend framework	Express 4	Minimal routing + middleware; no framework lock-in
Database	Supabase (managed Postgres)	Postgres with PostGIS and pgcrypto available; hosted, no ops
DB access	@supabase/supabase-js (PostgREST)	No ORM layer to maintain; raw SQL still available via RPC
Geospatial	PostGIS (geography, 4326)	Radius search done in the database, not in JS
Auth	jsonwebtoken + bcryptjs	Stateless tokens; bcrypt cost factor 10
Config	dotenv with fail-fast validation	Missing env var crashes at boot, not at first request

3. Architecture

3.1 Shape

Three tiers, strictly separated. The browser never talks to the database.

```

Browser (React SPA, Vite)
|  HTTPS, JSON, Authorization: Bearer <JWT>
v
Express API (Node)  |-- authMiddleware --> requireRole --> controller
|  supabase-js, service_role key
v
Supabase Postgres (+ PostGIS, pgcrypto, 3 SQL RPCs)

```

Why the API tier exists at all. Supabase can be called directly from the browser. It was deliberately not done that way: doing so would push authorization into Row Level Security policies and expose the database surface to the client. Routing everything through Express keeps one enforcement point, keeps the database credential server-side, and leaves the option to swap Supabase for any Postgres later without touching the frontend.

3.2 Request lifecycle

- axios interceptor in src/lib/api.js reads the token from localStorage and sets the Authorization header on every request.
- authMiddleware verifies the JWT signature, then re-loads the user row from the database by sub. The token is not trusted for role data - the role is read fresh each request, so an admin demoting a user takes effect immediately rather than at token expiry.
- requireRole(...roles) is a small factory returning a middleware; it 403s anything outside the allowed set.
- The controller performs the query and returns { data } or { error }. Response envelope is uniform across every endpoint.

3.3 Code layout

Path	Contents
backend/src/config	env validation, Supabase client singleton
backend/src/middlewares	auth.middleware.js, role.middleware.js
backend/src/routes	One router per resource: auth, events, me, rewards, admin
backend/src/controllers	Grouped by resource, then split by HTTP verb (getRequests, postRequests, updateRequests, deleteRequest)
backend/src/utlis	jwt.js (sign/verify), password.js (hash/compare)
backend/sql	schema.sql, setup.sql (schema + demo accounts), idempotent migrate_*.sql, seed_*.sql
src/pages	Split by role: auth/, user/, organizer/, admin/
src/layouts	UserLayout (header + bottom nav), DashboardLayout (sidebar, used by both portals)
src/routes	ProtectedRoute - the single client-side guard

4. Data model

Four tables. All primary keys are uuid defaulting to gen_random_uuid(). Every status column is a check constraint rather than an enum type, so values can be widened by a migration without an ALTER TYPE.

Table	Key columns
users	name, email (unique), password (bcrypt hash), role (user organizer admin), organizer_status (none pending approved rejected), img_url, created_at
events	title, description, event_category (Sports Lifestyle Community Networking Others), address, location (geography point 4326), latitude, longitude, start/end date and time, capacity, status (submitted cancelled completed), fee_type (free paid), reward_type, modality, approval_status (pending approved rejected changes_requested), organizer_id, approved_by, img_url
event_registrations	event_id, user_id, registration_date, status (registered waitlisted cancelled), unique (event_id, user_id)
rewards	user_id, event_id (nullable, ON DELETE SET NULL), type, amount, status (pending granted)

Referential policy is intentional per relation. Deleting an event cascades its registrations - a registration has no meaning without its event. Deleting an event only nulls the link on rewards, because a reward already granted is a record of something the employee actually did and must survive.

Indexes cover the real query paths: events by organizer and by approval_status, registrations by event and by user, rewards by user.

4.1 Stored procedures (RPC)

Three things PostgREST cannot express, pushed into SQL functions:

- `nearby_events(lat, lng, radius_m)` - ST_DWithin filter on approved events with a location, ordered by ST_Distance. Radius search in the database; the client never downloads events to filter them.
- `leaderboard_rewards()` - sum of granted reward amounts grouped by user, ordered descending.
- `monthly_registrations()` - registrations grouped by `date_trunc('month', ...)` for the admin analytics chart.

5. API surface

All routes are prefixed `/api`. "Auth" means a valid Bearer token is required.

Endpoint	Purpose	Access
GET <code>/api/health</code>	Liveness probe	public
POST <code>/api/auth/register</code>	Create account, returns token	public
POST <code>/api/auth/login</code>	Exchange credentials for token	public
GET <code>/api/auth/me</code>	Current user from token	auth
GET <code>/api/me</code>	Current user (used by ProtectedRoute)	auth
GET <code>/api/me/registrations</code>	Own registrations, event embedded	auth
GET <code>/api/me/rewards</code>	Own rewards + granted total	auth
POST <code>/api/me/apply-organizer</code>	Raise organizer application	auth, role user
GET <code>/api/events</code>	List, filterable by category / modality / title search	auth
GET <code>/api/events/nearby</code>	Radius search (PostGIS RPC)	auth
GET <code>/api/events/:id</code>	Single event + taken count	auth
POST <code>/api/events</code>	Create event (submitted / pending)	organizer
PATCH <code>/api/events/:id</code>	Edit own event	organizer (owner)
PATCH <code>/api/events/:id/status</code>	Approve or reject	admin
POST <code>/api/events/:id/register</code>	Join an approved event	role user
DELETE <code>/api/events/:id/register</code>	Cancel registration (soft)	role user
GET <code>/api/events/:id/registrations</code>	Attendee list	admin, or owning organizer
GET <code>/api/leaderboard</code>	Reward leaderboard	auth
GET <code>/api/admin/users</code>	All users	admin
PATCH <code>/api/admin/users/:id</code>	Edit name, role, organizer_status, img_url	admin
GET <code>/api/admin/analytics</code>	Event and registration totals, by month	admin

6. Authorization model

Authorization is enforced at three depths, because role alone is not enough:

- **Authentication** - `authMiddleware`, mounted with `router.use()` at the top of every protected router. A route added later is guarded by default; nothing has to be remembered per route.
- **Role** - `requireRole('organizer')` and friends on the specific routes.
- **Ownership** - role is not sufficient for organizer routes. PATCH `/events/:id` loads the row and 403s if `organizer_id` is not the caller; GET `/events/:id/registrations` does the same, while still letting an admin through. Without this, any organizer could edit any other organizer's event.

Query-level scoping. `listEvents` narrows by role inside the query itself: a user sees only `approval_status = 'approved'`. Unapproved events are not filtered out in the browser - they never leave the database.

7. Implementation decisions worth defending

- **Registration cancel is a soft state change, not a delete.** Status moves to cancelled and the row stays. That preserves history, and combined with the unique (`event_id`, `user_id`) constraint it means re-joining must be an upsert with `onConflict: 'event_id,user_id'` - a plain insert would violate the constraint.
- **Remaining capacity is computed by the database.** Event queries select `*`, `event_registrations(count) filtered to status = 'registered'`, so the taken count arrives with the event in one round trip. No N+1, and no denormalised counter that can drift out of sync.
- **Role is re-read on every request, not taken from the JWT.** The token carries only `sub` and `role`; the middleware re-queries the user. Slightly more work per request, but privilege changes apply instantly.
- **The client re-syncs the user on entering any protected route.** `ProtectedRoute` calls `/me` and replaces the cached user, so an approved organizer application appears without the user logging out and back in. A 401 there clears local state.
- **Coordinates are stored twice, on purpose.** `latitude/longitude` as plain doubles for the client to read, plus a PostGIS geography column written as `SRID=4326;POINT(lng lat)` for the distance query. Note the order inversion - PostGIS takes longitude first.
- **Setting a role manually clears any open application.** If an admin changes `role` without passing `organizer_status`, the status is reset to `none`, so no user is left in a contradictory "organizer, still pending" state.
- **Env vars fail fast.** `required()` throws at import time. A missing key stops the process at boot with a named error rather than producing confusing 500s in production.
- **Cover images are size-capped.** Covers arrive as base64 data URLs, so the JSON body limit is raised to 4 MB and `createEvent` rejects an `img_url` over 2 MB with a 413.

8. Known limitations and the planned fix

Stating these before being asked is stronger than being caught by them.

Limitation	Fix when it matters
The backend uses the Supabase <code>service_role</code> key, which bypasses Row Level Security entirely. All authorization lives in Express.	Acceptable while the API is the only client. Before any direct browser-to-Supabase access, RLS policies become mandatory.
JWT is stored in <code>localStorage</code> , readable by any injected script (XSS).	Move to an <code>httpOnly</code> , <code>SameSite</code> cookie with a CSRF token.
No refresh tokens; the 1-day token cannot be revoked before expiry.	Short access token + refresh token, or a server-side deny list.
No rate limiting on login - brute force is only slowed by bcrypt cost.	<code>express-rate-limit</code> keyed on IP and email.
capacity is stored and displayed but not enforced server-side; the waitlisted status exists unused.	Enforce in a transaction or a DB trigger - checking in JS races under concurrent joins.
Rewards are inserted by seed data; there is no automated grant on attendance.	Grant on an organizer marking attendance, or on event completion.
No pagination on list endpoints.	Range headers via PostgREST once volume justifies it.
Base64 cover images live in a text column, bloating rows and responses.	Supabase Storage; keep only the object URL in the row.
No automated test suite.	Integration tests over the API against a throwaway schema give the most coverage per line written.
<code>login()</code> collapses a query error and a missing user into the same 401, which hides real database faults during debugging.	Distinguish error from a null row; log the former, return 401 only for the latter.

9. Questions to be ready for

Why a custom Express API instead of calling Supabase from the browser?

To keep one authorization boundary. Direct access would move every rule into RLS policies and put the database credential in the client. The API also keeps the frontend portable to any Postgres host.

Why not Supabase Auth, given you use Supabase?

The user table needs role and organizer_status as first-class, admin-editable columns, and every route already goes through the API. Custom JWT plus bcrypt keeps the identity model in one place. The auth_user_id column is already present as the migration path if Supabase Auth is adopted later.

Why no ORM?

Four tables and PostgREST cover the CRUD. An ORM would add a schema definition to keep in sync with the SQL for no gain at this size, and the three genuinely complex queries are SQL functions anyway.

How does a user become an organizer?

POST /api/me/apply-organizer sets organizer_status to pending. An admin sets role and status via PATCH /api/admin/users/:id. The next protected route entry re-syncs the client from /me, so the portal appears without a re-login.

How does an event become visible?

Created as status = 'submitted', approval_status = 'pending'. An admin sets approved or rejected, which also records approved_by. Only approved events reach a user, and registration re-checks approval, so an event rejected between page load and click cannot be joined.

How does map search work?

The nearby_events RPC runs ST_DWithin against a geography column and orders by distance. Filtering in Postgres rather than in JavaScript means the payload is already scoped to the radius.

What stops one organizer editing another's event?

An ownership check inside the controller, not just the role middleware. The row is loaded and organizer_id compared to req.user.id before any write.

What happens if a user cancels and rejoins?

The row is reused. Cancel sets status = 'cancelled'; rejoining upserts on the unique (event_id, user_id) pair back to registered. A plain insert would violate the constraint.

How are passwords stored?

bcrypt at cost 10 via bcryptjs, hashed in the API. Seeded demo accounts use Postgres crypt(..., gen_salt('bf', 10)), which produces a compatible hash. The password column is stripped from every response before it is sent.

What is the biggest security gap today?

The service_role key. It is full database admin and bypasses RLS, so a compromised API process is a compromised database. It is kept out of version control, and any key that has been copied between machines should be rotated in the Supabase dashboard.

Does this scale?

To the intended population - one company's employees - comfortably, since the heavy work (distance search, aggregation, counting) is already in Postgres with indexes on the real query paths. The first things to break are the missing pagination and the base64 images, both listed above with their fixes.

How would you deploy it?

Static frontend build to any CDN host; the Express API as a Node service with the env vars set in the platform, not in a file; CORS_ORIGIN pinned to the deployed frontend origin rather than *; Supabase already managed.

What would you build next?

Server-side capacity enforcement with the waitlist that the schema already anticipates, then automated reward granting on attendance - together they close the loop from joining an event to appearing on the leaderboard.